Benny Wen
Professor Kurt Brown
CSC 59866
18 October 2026

Duplicate Actions After Lost Responses in Environmental Reporting APIs

What Problem Are You Investigating?

API's (application programming interface) is the way one program asks another to do something. Clicking "submit" on a website often triggers an API request behind the scenes or requesting data from a server. This project focuses on one specific failure, the lost response. When a program sends a request, the server executes that request, but the "success" message never makes it back to the client. The program has no way of knowing whether the job was completed, so a careful program tries again. If the server does not realize it is receiving the same request twice, it does the job twice, wasting resources server side and client side too.

Status codes are specific error or success codes that servers send back to a client after it's done executing the request. Different codes correspond to different messages, like error server down, data not found, etc. However this does not apply to our problem, our project specifically focuses on when the response gets lost, the program never receives a status code at all. ALl it gets is a timeout or connection error, which signals that something went wrong but not whether the server finished the request. This is the missing answer at the core of our problem.

Who or What Is Affected By It?

This problem affects anyone who builds, runs, or depends on an API that creates or changes records. Different groups are affected in different ways depending on the task they are doing.

Software engineers and developers who build and maintain API services have to write the retry, duplicate checking, and status tracking logic. Oftentimes having to test it against failures that are hard to reproduce, having to debug problems that leave no error on the server side, since in our scenario, the word has completed server side. Engineers who build client apps are similarly affected, calling an API requires the engineer to decide whether to retry after a timeout, which means choosing between risking a duplicate and risking an action that was never completed. Data analysts and agency staff who rely on the records for reporting, compliance, or decisions are affected when duplicate or inconsistent entries skew totals or trigger repeated notifications and calculations. The general public who uses published information from public or even private databases via APIs are affected downstream if those records are wrong.

In other words, anyone who interacts with the modern web who is using/maintaining a service that calls an API with this problem is affected and the severity depends on their role (user, engineering, analyst, etc.).

What Evidence Do You Currently Have That It Is a Real Problem?

Request for proposals P2027-03 from the South Coast AIr Quality Management District (AQMD) seeks contractors to build and support the agency's software, which includes Web APIs, API testing, and rollback (undoing a bad change). It counts 83 web services and 65 web applications, 43 of them which are public facing (*Software Systems*).

The RabbitMQ reliability guide, a guide for software that passes messages between programs, explains that confirmations can be lost, so senders resend and duplicates become possible. Receiving programs must either filter duplicates or handle them safely (*Reliability Guide*).

RFC 9110, the HTTP standard, calls a request safe to repeat (idempotent) if sending it several times has the same effect as sending it once. POST, the request type that creates records, is not idempotent, so retrying it is not safe by default (*Fielding et al.*). Clients are assumed to retry automatically after a timeout, and duplicates are assumed to cause downstream problems. Neither has been verified for a real agency system.

What Solutions Already Exist?

There are currently two existing approaches to this problem. One approach is to answer right away, with a retry key. The server handles the request and replies immediately. TO make retries safe, the client attaches a unique ID to each submission. The server saves that ID along with the result. If the same ID shows up again, the server just sends back the saved result instead of doing the work all over again. This builds on the idea from RFC 9110 that repeating a request should have the same effect as sending it once (*Fielding et al.*). The downside to this approach is that the server has to remember IDs long enough to cover realistic retries, which leads to wasted memory storage.

The second approach is to put the work in a queue and track the job status. The server accepts the request, puts the work in a queue and replies "accepted, job x". The server then carries out the request, and the client can check the status record to see if it has finished. RabbitMQ supports this approach while acknowledging that its guide recommends durable queues and persistent messaging for important data (*Reliability Guide*). This approach is slow but reliable. The guide says delivery is at least once, so duplicates are still possible and the worker has to be safe to repeat (*Reliability Guide*). Additionally, RabbitMQ's guarantees cover delivery between the queue and the program using it. However it does not automatically tell an HTTP client whether the job has finished, so the status record would have to be designed as part of this project.

The more traditional method to this problem is just to retry. The client resends after a timeout, and the server treats every request as a brand new one. But as mentioned earlier, this leads to downstream problems.

What Is the Most Important Thing You Still Do Not Know?

The biggest unknown is after a response is lost, how can the client reliably tell whether the job has finished? What does each approach cost to find that out?

A controlled experiment on a mock reporting service could potentially answer this. A small piece of code would intentionally discard the response after the server has saved the report, for a fixed share of request (20%). Then 100 generated reports would be sent through each of the three approaches, with the client allowed up to three retries, and then we record (1) how many duplicate records get created, (2) how many submissions never complete, (3) how many retires each submission needs, and (4) how long it takes until the client has a clear, confirmed answer.

The results could potentially change how the project is approached. If Approach A gives zero duplicates but clients get confused when old IDs expire, a status lookup should be added next to the IDs. If approach B's status record answers the question but creates duplicates when restarted, then the focus would shift to making the worker safe to repeat.

Individual Scope

This project will build a mock environmental reporting service that accepts a report submission, saving it as a record, and replying with a confirmation using only invented data on a local machine. The service will be written in Python with a local database and a small piece of the code will deliberately discard the responses after the report has been saved for about 20 percent of the requests.

Three versions of the service will then be compared on the same set of 100 generated reports, a baseline that retries and treats every request as new. One which attaches a unique retry ID so repeated requests return the saved result. And the last approach which uses a RabbitMQ queue and a stored job status the client can check.

For each version, the data that will be recorded follows as mentioned in the previous section. This comparison is meant to show how a client can reliably tell whether a job finished after a lost response and what each approach costs to find out. The idea to be tested is that combining retry IDs with a stored status that the client can check removes duplicates and gives a clear answer. The project is manageable for one student because it involves a single workflow, one type of failure and three different approaches that aren't too complex to implement. The results are measured quantitatively so the project should be challenging but not overwhelming.

Works Cited

Fielding, Roy T., et al., editors. *HTTP Semantics*. RFC 9110, Internet Engineering Task Force, June 2022, https://www.rfc-editor.org/rfc/rfc9110. Accessed 15 Oct. 2026.

"Reliability Guide." *RabbitMQ Documentation*, version 4.3, Broadcom, https://www.rabbitmq.com/docs/reliability. Accessed 15 Oct. 2026.

Software Systems Development, Maintenance and Support Services. RFP P2027-03, South Coast Air Quality Management District, 4 Sep. 2026, https://www.aqmd.gov/docs/default-source/bids/p2027-03.pdf. Accessed 15 Oct. 2026.